Fase 2 — Identificación de clases y responsabilidades

A partir del análisis de la Fase 1 identificamos 20 tarjetas CRC y 3 enumeraciones. Este documento muestra el ejercicio de sustantivos y verbos sobre el enunciado, las tarjetas CRC resultantes y por qué cada clase existe. Es la base directa del diagrama UML v1 (Fase 3).

1. Técnica de sustantivos y verbos

Sobre el texto del enunciado marcamos los **sustantivos** (candidatos a clase o atributo) en negrita y los *verbos* (candidatos a método) en cursiva.

> Una **universidad** ha adquirido una **flota** mixta de **robots** autónomos de reparto para *transportar* **objetos** dentro del **campus**: **documentos** entre **facultades**, **alimentos** desde la **cafetería**, **materiales de laboratorio** y **paquetes** de la **oficina de correspondencia**. Existen distintos tipos de robots (**terrestres de ruedas**, **terrestres de orugas** para **zonas verdes** y **drones** para **azoteas**), cada uno con reglas propias de **carga máxima**, **velocidad**, **autonomía de batería** y **restricciones de zona**.
>
> El sistema *registra* los robots disponibles y su **estado** (**batería**, **ubicación**, **carga actual**). *Recibe* **solicitudes de entrega** con distintos tipos de carga (documentos, alimentos con **control de temperatura**, **frágiles**, **materiales peligrosos**). *Asigna* una solicitud a un robot compatible según **reglas de negocio**. *Calcula* **costos** y **tiempos estimados**. *Consulta* **condiciones externas** (**clima**) mediante una **API pública** antes de *autorizar* entregas con drones. *Lleva* una **bitácora** de **eventos** que pueda *mostrarse* como **reporte**.

Decisión por cada sustantivo

Sustantivo	Decisión	Motivo
robot	Clase abstracta `Robot`	Concepto central con comportamiento común a todos los tipos.
terrestre de ruedas, de orugas, dron	Subclases `RobotRuedas`, `RobotOrugas`, `Dron`	Cada uno cambia velocidad, zonas, costo y si necesita clima: comportamiento distinto, no solo datos.

Sustantivo	Decisión	Motivo
flota	Clase `Flota`	Agrupa robots y responde qué robots están disponibles.
batería, autonomía de batería	Clase `Bateria`	Tiene reglas propias (0–100 %, consumo, reserva mínima).
ubicación, facultad, cafetería, azotea, zona verde	Clase `Ubicacion` + enum `Zona`	La ubicación calcula distancias; la zona es un conjunto cerrado de valores.
carga máxima, velocidad	Atributos de `Robot`	Datos simples, sin comportamiento propio.
estado	Enum `EstadoRobot`	Conjunto cerrado: disponible, en ruta, mantenimiento.
objetos: documentos, alimentos, frágiles, materiales peligrosos, paquetes	Clase abstracta `Carga` + `Documento`, `Alimento`, `Fragil`, `MaterialPeligroso`	Cada tipo tiene su propia restricción y su recargo. Un paquete se trata como `Fragil` o `Documento` según su contenido.
control de temperatura	Atributo de `Alimento` y capacidad del robot	Es una restricción de la carga, no una entidad.
solicitud de entrega	Clase `SolicitudEntrega`	Tiene ciclo de vida: pendiente, asignada, entregada o fallida.
reglas de negocio	Clase `AsignadorEntregas`	Coordina flota, clima y bitácora; ponerlo en `Flota` mezclaría responsabilidades.
costo, tiempo estimado	Métodos de `Robot`	Varían por subtipo: son comportamiento polimórfico, no clases.
clima, condiciones externas, API pública	Interfaz `ProveedorClima` + `ServicioClimaOpenMeteo` + `ServicioClimaSimulado`	El PRD exige aislar la API y poder sustituirla.
umbral de clima	Junto con el reporte: `ReporteClima / UmbralesClima`	Guarda los datos del clima y los límites del supuesto S1.

Sustantivo	Decisión	Motivo
bitácora, evento	Clases `Bitacora` + `EventoBitacora` + enum `TipoEvento`	La bitácora crea y guarda sus eventos.
reporte	Método de `Bitacora`	Es una vista de los eventos, no tiene estado propio.
universidad, campus	Fuera del modelo	Son el cliente y el contexto; no tienen comportamiento en el sistema.

2. De los verbos a las responsabilidades

Cada verbo del enunciado se volvió un método en la clase que tiene la información para hacerlo.

Verbo	Método	Clase	Por qué ahí
registrar robots	`agregar(robot)`	`Flota`	La flota es la que conoce a todos los robots.
recibir solicitudes	`asignar(solicitud)`	`AsignadorEntregas`	Es el punto de entrada de una solicitud nueva.
asignar a un robot compatible	`puede_atender(solicitud)`	`Robot`	Solo el robot sabe su peso máximo, sus zonas y su batería.
asignar (elegir)	`asignar(solicitud)`	`AsignadorEntregas`	Elegir entre varios robots requiere ver toda la flota.
calcular costos y tiempos	`calcular_costo()`, `calcular_tiempo_min()`	`Robot`	Cambian según el tipo: polimorfismo.
consultar condiciones externas	`condiciones_favorables(ubicacion)`	`ProveedorClima`	Aísla la API del resto del sistema.
autorizar / rechazar / reasignar	dentro de `asignar()` y `reintentar_pendientes()`	`AsignadorEntregas`	Son decisiones de coordinación.
transportar, entregar	`cargar(carga)`, `entregar(destino)`	`Robot`	Cambian el estado y la batería del propio robot.

Verbo	Método	Clase	Por qué ahí
llevar bitácora	`registrar(tipo, mensaje)`	`Bitacora`	La bitácora crea y guarda los eventos.
mostrarse como reporte	`generar_reporte()`	`Bitacora`	El reporte se arma a partir de los eventos que ella guarda.

3. Tarjetas CRC

Hay 20 tarjetas, agrupadas en robots, cargas, entregas, clima y bitácora. Cada una dice qué sabe y qué hace la clase (responsabilidades) y con quién necesita trabajar (colaboradores).

Robots

Robot (abstracta) — Responsabilidades	Colaboradores
Conocer su id, carga máxima, velocidad, ubicación y estado	`Bateria`, `Ubicacion`
Decidir si puede atender una solicitud (peso, zonas, carga compatible, batería)	`SolicitudEntrega`, `Carga`, `Bateria`
Calcular tiempo, costo y consumo de una entrega (cada subtipo a su manera)	`Ubicacion`, `Carga`
Indicar si requiere consultar el clima	—
Cargar, entregar e ir a mantenimiento	`Bateria`, `Carga`

RobotRuedas / RobotOrugas / Dron — Responsabilidades	Colaboradores
Definir sus zonas accesibles, costo por km, consumo por km y factor de ruta	`Zona`
Dron: declarar que es aéreo y que requiere clima	—
Orugas: declarar terreno irregular (no apto para frágiles)	—

Bateria — Responsabilidades	Colaboradores
Mantener el nivel entre 0 y 100 %	—
Consumir y recargar	—
Responder si alcanza para un consumo dejando la reserva mínima	—

Flota — Responsabilidades	Colaboradores
Agregar, quitar y buscar robots	`Robot`
Listar los robots disponibles	`Robot`

Ubicaciones y cargas

Ubicacion — Responsabilidades	Colaboradores
Conocer nombre, coordenadas y zona	`Zona`
Calcular la distancia a otra ubicación (Haversine)	—

Carga (abstracta) → Documento, Alimento, Fragil, MaterialPeligroso — Responsabilidades	Colaboradores
Conocer descripción y peso (validado > 0)	—
Dar su recargo sobre el costo (cada subtipo distinto)	—
Decidir si es compatible con un robot según las capacidades del robot	`Robot`

Entregas

SolicitudEntrega — Responsabilidades	Colaboradores
Conocer origen, destino y carga	`Ubicacion`, `Carga`
Llevar su estado y el motivo si queda pendiente o falla	—

SolicitudEntrega — Responsabilidades	Colaboradores
Guardar robot asignado, costo y tiempo estimado	`Robot`

AsignadorEntregas — Responsabilidades	Colaboradores
Elegir el robot compatible más barato para una solicitud	`Flota`, `Robot`, `SolicitudEntrega`
Consultar el clima antes de asignar un dron	`ProveedorClima`
Reasignar o dejar pendiente; reintentar pendientes	`SolicitudEntrega`
Completar entregas y registrar todo en la bitácora	`Bitacora`, `Robot`

Clima

ProveedorClima (interfaz) — Responsabilidades	Colaboradores
Entregar un reporte de clima para una ubicación	`Ubicacion`, `ReporteClima`
Decir si las condiciones son favorables para volar	`ReporteClima / UmbralesClima`

ServicioClimaOpenMeteo — Responsabilidades	Colaboradores
Hacer la petición HTTP a Open-Meteo y traducir el JSON a un reporte	`ReporteClima`
Manejar timeout y errores devolviendo un reporte "no disponible"	—

ServicioClimaSimulado — Responsabilidades	Colaboradores
Devolver valores de clima configurables (pruebas y demo)	`ReporteClima`

ReporteClima / UmbralesClima — Responsabilidades	Colaboradores
Guardar viento, ráfagas, probabilidad de lluvia, día o noche, temperatura y disponibilidad	—

ReporteClima / UmbralesClima — Responsabilidades	Colaboradores
Evaluar el reporte contra los umbrales y explicar el motivo	—

Bitácora

Bitacora — Responsabilidades	Colaboradores
Crear y guardar eventos (fecha, tipo, mensaje, robot)	`EventoBitacora`, `TipoEvento`
Filtrar eventos y generar el reporte de texto o CSV	`EventoBitacora`

EventoBitacora — Responsabilidades	Colaboradores
Conocer fecha, tipo, mensaje y robot involucrado	`TipoEvento`

Enumeraciones: `Zona` (edificio, zona verde, azotea), `EstadoRobot` (disponible, en ruta, mantenimiento) y `TipoEvento` (entrega exitosa, entrega fallida, rechazo por clima, solicitud pendiente, mantenimiento).

4. Cómo se llegó a cada clase

La mayoría de las clases salió directo de un sustantivo; las más discutidas fueron `Bateria`, `AsignadorEntregas` y `ProveedorClima`.

Robot y sus subtipos. La primera idea fue una sola clase `Robot` con un atributo `tipo` y varios `if`. La descartamos porque cada regla nueva (velocidad, zonas, clima) obligaba a editar todos los `if`. Un dron *es un* robot, así que la herencia es real: el sistema trata a todos como `Robot` y cada subtipo responde a su manera.

Bateria. Empezó como un atributo `float` de `Robot`. Al escribir las reglas (no bajar de 0 %, no pasar de 100 %, reservar 15 %) vimos que esas validaciones se repetían en el robot y en el asignador. Por eso pasó a ser una clase que se protege sola.

Ubicacion y Zona. "Facultad", "cafetería" y "azotea" parecían clases distintas, pero solo cambian el nombre y la zona. Una sola clase `Ubicacion` con un enum `Zona` cubre todos los puntos del campus y además calcula distancias.

Carga y sus subtipos. Pensamos en un enum `TipoCarga` con las reglas dentro del asignador. Lo descartamos porque el asignador terminaría conociendo las reglas de cada carga. Con subclases, cada carga decide si es compatible con un robot y cuánto recargo tiene; agregar una carga nueva no toca nada más.

SolicitudEntrega. No es un simple registro: pasa por estados (pendiente, asignada, entregada, fallida) y guarda el motivo cuando algo sale mal. Por eso es una clase con su propio ciclo de vida.

Flota. Al principio pensábamos poner ahí la asignación. Lo separamos porque la flota solo debe saber qué robots hay; si también consultara el clima y escribiera en la bitácora, tendría demasiadas responsabilidades.

AsignadorEntregas. Nació de "reglas de negocio" y del verbo "asignar". El robot sabe si *puede* atender una solicitud, pero elegir entre varios robots y coordinar clima y bitácora necesita una clase propia.

ProveedorClima y sus implementaciones. El PRD pide poder cambiar el proveedor de clima sin tocar la asignación. Una clase concreta `ServicioClima` aislaba la API, pero el asignador quedaba atado a ella. Con una interfaz, el asignador depende de `ProveedorClima` y da igual si detrás está Open-Meteo o el simulador.

ReporteClima / UmbralesClima. Necesitábamos un objeto que llevara los datos del clima al asignador sin exponer el JSON de la API. En esta fase lo pensamos como una sola tarjeta junto con los umbrales del supuesto S1.

Bitacora y EventoBitacora. La bitácora crea sus propios eventos para que nadie de afuera pueda alterar el historial. El reporte es un método y no una clase, porque solo muestra los eventos que ya existen.

5. Qué queda fuera del modelo

Dejamos fuera todo lo que no ayuda a los objetivos de POO del proyecto o que el enunciado no pide.

Elemento	Motivo
Universidad y campus como clases	Son el cliente y el contexto; no tienen comportamiento en el sistema.
Usuarios, login y permisos	El enunciado no pide autenticación y no aporta a los objetivos de POO.
Navegación real del robot (sensores, obstáculos)	El sistema es de gestión, no controla el hardware.

Elemento	Motivo
Base de datos	Se trabaja en memoria; la bitácora se exporta a archivo.
Rutas reales por senderos (OSRM)	Opcional; se aproxima con Haversine y un factor de rodeo. Si se agrega, irá detrás de su propia interfaz.
Facturación a las facultades	Solo se calcula el costo estimado.
Estaciones de carga físicas	Se simplifican como "mantenimiento", que recarga la batería.
Interfaz gráfica	No es parte del dominio: es una capa aparte que solo usa `AsignadorEntregas`, `Flota` y `Bitacora`.